Лабораторна робота №5

Імпорт модулів та пакетів. Робота із файлами.

Мета роботи: імпорт модулів та пакетів. Робота із файлами. Генератори, функції-генератори. Робота з вбудованими функціями: `map`, `filter`, `zip`, `all` та `any`. Сортування за ключом методом `sort` та функцією `sorted`.

Теоретичні відомості

1. Зазначимо кілька вельми корисних функцій, що часто використовуються. І почнемо із функції `map`. Вона дозволяє перетворювати елементи об'єкта, що ітерується, у відповідності з деякою зазначеною функцією:

```
map(func, *iterables)
```

Пояснимо роботу цієї функції на такому простому прикладі. Припустимо, що ми маємо список і ми хочемо отримати список із квадратів цих чисел:

```
lst = [1, -2, 3, -4]
```

Для цього створимо функцію зведення числа у квадрат:

```
def sqr(x):
    return x**2
```

Тепер викличемо функцію map:

```
squares = map(sqr, lst)
```

Зверніть увагу, ми передаємо посилання на функцію, тобто записуємо її ім'я без круглих дужок. В результаті змінна `squares` посилатиметься на ітератор `map` і ми з попередньої лабораторної роботи знаємо, що для перебору всіх значень можна скористатися функцією `next`:

```
print(next(squares))
print(next(squares))
print(next(squares))
```

Або сформувати новий список за допомогою функції `list`:

```
squares_lst = list(squares)
print(squares_lst)
```

Тобто функція map до кожного елемента списку lst застосовує функцію sqr і повертає об'єкт-генератор для послідовного вибору нових, перетворених значень. У певному сенсі, це еквівалент такого виразу-генератора:

```
squares = (sqr(value) for value in [1, -2, 3, -4])
```

або ось такого списку:

```
squares_lst = [sqr(1), sqr(-2), sqr(3), sqr(-4)]
```

Причому, у всіх випадках функція sqr, або будь-яка інша повинна приймати лише один аргумент. Якщо замість функції sqr вказати функцію, що приймає 2 аргументи, то виникне помилка:

```
def power(x, n):
    return x**n
powers = map(power, lst)    # помилка
```

А ось повертати вона може кілька аргументів:

```
def value_and_sqr(x):
    return x, x**2
```

В результаті отримаємо список кортежів:

```
lst = [1, -2, 3, -4]
result = list(map(value_and_sqr, lst))
print(result)    # [(1, 1), (-2, 4), (3, 9), (-4, 16)]
```

Подібні перетворення можна виконувати з будь-якими типами даних, наприклад рядками:

```
lst = ["Київ", "Харків", "Полтава"]
print(*map(len, lst)) # 4 6 7
```

Тут замість перетворення на список, ми використали оператор розпаковки.

Якщо потрібно застосувати вбудовані рядкові методи, то такий метод треба вказати із відповідним простором імен:

```
lst = ["Київ", "Харків", "Полтава"]
print(*map(str.upper, lst))     # КИЇВ ХАРКІВ ПОЛТАВА
```

Досить часто першим аргументом функції map вказують анонімні (лямбда) функції:

```
lst = ["Київ", "Харків", "Полтава"]
print(*map(lambda x: x[::-1], lst))
                        # вїиК вікраХ аватлоП
```

Як бачимо, результатом є рядки, записані у зворотньому напрямку.

Оскільки функція `map` другим аргументом приймає будь-який об'єкт, що ітерується, то ми можемо результат роботи однієї функції `map` використовувати у іншій, зовнішній функції `map`:

```
lst = ["Київ", "Харків", "Полтава"]
result = map(lambda x: x.replace("a", "A"), lst)
print(*result)    # Київ ХАрків ПолтАвА
```

Дуже часто в програмах Python функцію `map` використовують для зчитування з клавіатури кількох чисел через будь-який роздільник, наприклад, пробіл. Це робиться так:

```
number_list = list(map(int, input().split()))
```

Спочатку запускається функція `input`, яка повертає рядок з числами через пробіл. Метод `split` перетворює цей рядок на список чисел, але ці числа все ще існують як рядки (об'єкти типу `str`). Функція map застосовує до кожного такого рядка функцію `int` і в результаті ми отримуємо map-об'єкт, який містить числа типу `int`. Нарешті, функція `list` конвертує map-об'єкт у список, який містить звичайні числа типу `int`.

Функція filter

Наступна функція — `filter`. Сама її назва каже, що вона певним чином фільтрує значення за якимось критерієм. Цей критерій — функція, посилання на яку передається першим аргументом. Другий аргумент — ітерований об'єкт, який має бути відфільтровано. Функція `filter` перебирає елементи другого аргументу, застосовує до них функцію-фільтр і залишає тільки ті значення, для яких фільтр повернув True. Загальний синтаксис:

filter(func, *iterables)

Припустимо, у нас є список:

```
lst = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

з якого потрібно вибрати тільки непарні значення. Для цього визначимо функцію перевірки на непарність:

```
def is_odd(x):
```

```
    return x % 2
```

І далі робимо виклик функції `filter`:

```
odds_only = filter(is_odd, lst)
print(*odds_only)    # 1 3 5 7 9
```

На виході отримуємо ітератор, який називається filter-об’єктом. Його можна передавати всюди, де приймаються ітератори.

Звісно, в ролі фільтру тут можна передавати лямбда-функції. Наш попередній приклад можна переписати так:

```
odds_only = list(filter(lambda x: x % 2, lst))
```

І це буває набагато зручніше, ніж оголошувати нову функцію.

Функцію `filter` можна використовувати з будь-якими ітерованими типами даних, наприклад, рядками. Нехай у нас є такий кортеж рядків, і ми хочемо залишити в ньому тільки такі рядки, які містять тільки літери. Ця задача розв’язується так:

```
lst = ("abc", "ab2", "123", "def", "r2d2")
result = filter(str.isalpha,lst)

for element in result:
    print(element, end=" ")       # abc def
```

Функція ZIP

Наступна корисна функція дозволяє об'єднувати між собою відповідні елементи впорядкованих колекцій. Наприклад, у нас є два списки:

```
a = [1,2,3,4]
b = [5,6,7,8]
```

І викликаючи для них функцію zip:

```
pairs = list(zip(a, b))
print(pairs)     # [(1, 5), (2, 6), (3, 7), (4, 8)]
```

Як бачимо, функція `zip` перетворила пару списків на ітератор, який містить кортежі пар, отриманих з елементів цих списків. Функція `zip` повертає zip-об’єкт, який далі може бути перетворений на довільну колекцію, або ж використаний там, де може бути оброблений ітератор.

Давайте тепер додамо ще один об'єкт, що ітерується – рядок:

```
c = "abracadabra"
```

І викликаємо функцію `zip` для всіх цих трьох об'єктів:

```
zipped = list(zip(a, b, c))
print(zipped)
```

В результаті отримаємо колекцію:

```
[(1, 5, 'a'), (2, 6, 'b'), (3, 7, 'r'), (4, 8, 'a')]
```

Ми отримали лише чотири кортежі, у кожному з яких по три елементи. Тобто всі символи рядка `"abracadabra"`, що залишилися, були просто відкинуті. Виходить, що функція `zip` формує вихідний список, довжина якого дорівнює довжині найменшої із зазначених колекцій. Якщо, наприклад, ми зменшимо колекцію a до двох елементів, то на виході також отримаємо список із двох елементів:

```
a = [1,2]
b = [5,6,7,8]
c = "abracadabra"
print(list(zip(a, b, c))) # [(1, 5, 'a'), (2, 6, 'b')]
```

Функції `map`, `filter`, `zip` є дуже популярними інструментами функціонального програмування у мові Python.

Завдання:

1. Написати програми у середовищі PyCharm відповідно до номера свого варіанту (кожне завдання оцінюється в 2 бали, максимум 8 балів за роботу).

Варіанти:

Варіант №1

Завдання №1. Імпортуйте у програму стандартний модуль `time` та, використовуючи однойменну функцію `time` з цього модуля, визначте час роботи програми, яка спочатку зберігає у файл з іменем `out.txt` рядок: `"Робота №5. Імпорт модулів та пакетів. Робота з файлами",` а потім читає з цього файлу його вміст і відображає прочитані дані на екрані. Також на екран потрібно вивести час роботи програми.

Завдання №2. Запишіть вираз-генератор, який повертає числа від 2 до 100000 з кроком 0.5 (тобто, 2.0, 2.5, 3, 3.5, ..., 100000). Виберіть перші п'ять значень цього генератора та відобразіть їх на екрані.

Завдання №3. Запишіть функцію-генератор для формування послідовності трибоначчі. Це послідовність чисел, у якій кожне наступне дорівнює сумі трьох попередніх, а перші два значення дорівнюють 0, 0 і 1. Наприклад, перші сім членів послідовності трибоначчі мають значення: 0, 0, 1, 1, 2, 4, 7. Вивести перші 15 чисел цієї послідовності, сформованої функцією-генератором.

Завдання №4. На вхід програми надходить список із дійсних чисел, записаних у рядок через пробіл. За допомогою функції `map` перетворити числа в рядку до типу `float` і відобразити перші три числа (вважається, що мінімум три числа наявні). Реалізувати отримання чисел через функцію `next`.

Варіант №2

Завдання №1. Імпортуйте в програму стандартний модуль `math` і, використовуючи функцію `log2`, обчисліть логарифм за основою 2 для цілих чисел від 1 до 10. Результат подайте у вигляді списку, який сформуйте за допомогою генератора списків. Отриманий список збережіть у файл у бінарному режимі доступу. Потім прочитайте його вміст в інший список. На екрані виведіть прочитані з файлу дані (останній список).

Завдання №2. Використовуючи символи малих літер латинського алфавіту (константа `ascii_lowercase`, яка може бути імпортована з модулю `string`), запишіть вираз-генератор, який повертав би всі можливі дволітерні комбаніції цих літер (`"aa", "ab"` тощо). Виведіть перші 50 комбінацій на екран.

Завдання №3. Є квадратична функція $f(x) = 0{,}1x^2 + 5x - 2$. Необхідно записати функцію-генератор, яка б видавала значення цієї функції в діапазоні $[a;\ b]$ з кроком 0,01, де a, b – це параметри функції-генератора. Викличте цю функцію зі значеннями $a = -5$, $b = 7$ і виведіть на екран перші 20 значень, які вона поверне. Будь-які колекції у програмі не використовувати.

Завдання №4. На вхід функції з іменем `dict_sort` надходить словник `dct`, виду:

```
dct = {'cat': 'кіт', 'horse': 'кінь', 'tree': 'дерево',
'dog': 'собака', 'book': 'книга'}
```

Необхідно відсортувати *значення* словника `dct` за зростанням *ключів* (лексикографічне сортування рядків) та повернути список із відповідних *значень* словника. Наприклад, для зазначеного словника `dct`, результатом має бути список: `['книга', 'кіт', 'собака', 'конь', 'дерево']`. Викличте функцію `dict_sort` для заданого в програмі довільного словника `dct` і виведіть повернутий нею список на екран.

Варіант №3

Завдання №1. Імпортуйте у програму стандартний модуль `random` та, використовуючи функцію `randint`, згенеруйте 100 випадкових чисел у діапазоні [-2; 5]. Всі ці числа слід подати у вигляді списку, сформованого за допомогою генератора списків. Збережіть отримані дані у файлі у текстовому режимі доступу. Потім прочитайте з нього дані і помістіть в інший список. Виведіть отриманий список на екран.

Завдання №2. Напишіть вираз-генератор, який повертав би добутки цифр послідовних натуральних чисел, від 1 до 100000. Виведіть на екран перші 50 значень.

Завдання №3. Створіть функцію-генератор, яка на основі символів рядка `chars = ascii_letters + digits + "!?@#$*"` формує та видає випадково згенеровані паролі довжиною n символів (n передається функції-генератору як аргумент, а до цього зчитується з клавіатури). Кількість паролів, що видаються функцією, повинна бути необмеженою. Виведіть перші п'ять згенерованих паролів на екран. Константи `ascii_letters` і `digits` можуть бути імпортовані з модулю `string`.

Завдання №4. На вхід програми надходить список цілих чисел, записаних в один рядок через пробіл. Необхідно вибрати з них чотири найбільші унікальні значення (тобто такі, які зустрічаються тільки один раз). Результат вивести на екран.

Варіант №4

Завдання №1. Імпортуйте в програму стандартний модуль `math` і, використовуючи функцію `log10`, обчисліть логарифм за основою 10 для цілих чисел від 1 до 10. Результат подайте у вигляді множини, який сформуйте за допомогою генератора множин. Отриманий список збережіть

файл у бінарному режимі доступу. Потім прочитайте вміст цього файлу і збережіть дані у множину (`set`). Виведіть на екран отриману множину.

Завдання №2. Напишіть вираз-генератор, який повертав би str-подання кодів Грея, починаючи з подання числа 1 до подання числа 255. Код Грея числа x може бути отриманий в результаті виконання операції XOR над двійковим поданням числа x і тим самим поданням, нециклічно зсунутим на один біт праворуч. Вивести на екран перші 8 кодів Грея.

Завдання №3. Визначте функцію-генератор, яка перебирала б усі *невпорядковані* комбінації із трьох цілих додатних чисел, кожне в діапазоні [2; 10] (*невпорядкованість* означає, що, наприклад, комбінації `(2, 3, 6)` і `(3, 2, 6)` ми не розрізняємо і видаватись має тільки одна подібна комбінація). Функція-генератор має повертати кортежі. За допомогою функції `filter` і описаного щойно генератора відберіть тільки ті трійки чисел, які можуть бути сторонами трикутника (довжина будь-якої сторони трикутника має бути меншою від суми двох інших сторін). Для роботи функції `filter` вам знадобиться функція-фільтр, яка приймає трійку чисел і повертає `True`, якщо ця трійка може бути довжинами сторін трикутника, а в протилежному випадку — `False`. Виведіть на екран перші 20 трійок. Будь-які колекції у програмі не використовувати (окрім кортежів трійками чисел).

Завдання №4. В зовнішньому файлі `matrix.txt` записана матриця M, елементами якої є цілі числа. Файл створіть самостійно. Приклад вмісту файлу:

```
2 4 7 1 6
7 0 2 7 5
8 4 2 6 1
```

Програма повинна прочитати цю матрицю у двовимірний кортеж. Після цього програма створює новий двовимірний кортеж, який містить транспоновану матрицю M^T. Відобразити нову матрицю на екрані. Транспонування виконати за допомогою функції `zip` без використання циклів.

Варіант №5

Завдання №1. Імпортуйте до програми стандартний модуль `time` і, використовуючи однойменну функцію `time` з цього модуля, визначте час роботи програми, яка спочатку зберігає у файл з ім'ям out.dat кортеж (у бінарному режимі доступу):

```
books = (('Сергій Жадан', 'Інтернат'), ('Ліна Костенко', 'Маруся Чурай'), ('Макс Кідрук', 'Колонія'))
```

а, потім, читає з цього файлу (також у бінарному режимі) його вміст та відображає прочитаний кортеж на екрані. Також на екран потрібно вивести час роботи програми.

Завдання №2. Є список із назв міст: `cities = ["Київ", "Харків", "Суми", "Умань", "Полтава", "Рахів"]`. Необхідно записати вираз-генератор, який використовуючи цей список, циклічно видає 1 000 000

найменувань міст (дійшовши до кінця списку, генератор має повертатись на початок списку і продовжувати далі). Вивести на екран перші 20 найменувань міст за допомогою створеного генератора.

Завдання №3. Запишіть функцію-генератор, яка видавала б кортежі довжиною n, що складаються із випадкових цілих чисел у діапазоні [a; b]. Величини n, a, b мають визначатися як параметри функції-генератора. Використовуючи створену функцію-генератор, сформуйте за допомогою генератора списків двовимірну колекцію розміром 20x20 елементів, що складається із випадкових чисел у діапазоні [-5;5]. Вивести отриману двовимірну колекцію на екран.

Завдання №4. Вводиться рядок зі слів, записаних через пробіл. Необхідно на їх основі скласти прямокутну таблицю із трьох стовпчиків. Якщо кількість слів не ділиться націло на 3, то «зайві» слова відкинути. Реалізувати цю програму за допомогою функції `zip`. Результат відобразити на екрані у вигляді прямокутної таблиці зі слів, записаних через пробіл.

Варіант №6

Завдання №1. Імпортуйте у програму стандартний модуль `random` та, використовуючи однойменну функцію `random`, згенеруйте 100 випадкових чисел. Усі ці числа слід подати у вигляді списку, сформованого за допомогою генератора списків. Збережіть отримані дані у файлі (у бінарному режимі доступу). Потім прочитайте з отриманого файлу дані і розмістіть в іншому списку. Виведіть отриманий список на екран.

Завдання №2. Створіть функцію-генератор, яка б необмежено повертала послідовні числа, записані у трійковій системі числення, починаючи з 1. Числа мають бути представлені як str-об’єкти. Виведіть на екран перші 20 згенерованих чисел.

Завдання №3. Створіть функцію-генератор, яка на основі символів рядка
`to_emails = ascii_letters + digits + "_"`
формує та видає випадково згенеровані email-адреси виду
`"xxxxxxxx@gmail.com"`,
де x - випадковий символ рядка `to_emails`. Кількість адрес, що видаються, повинна бути необмеженою. Виведіть перші сім згенерованих адрес на екран. Константи `ascii_letters` і `digits` можуть бути імпортовані з модулю `string`.

Завдання №4. Вводиться список цілих чисел в один рядок через пробіл. Необхідно залишити у ньому лише числа-паліндроми. Реалізувати програму за допомогою функції `filter`. Результат відобразити на екрані у вигляді послідовності чисел, що залишилися.

Варіант №7

Завдання №1. Імпортуйте у програму стандартний модуль `math` і, використовуючи функцію `factorial`, обчисліть факторіал для цілих чисел від 1 до 10. Результат подайте у вигляді словника, ключами якого є аргументи функції (числа від 1 до 10), а значеннями – обчислений факторіал. Сформуйте словник за допомогою генератора словників. Отриманий словник збережіть файл у текстовому режимі доступу (формат даних придумайте самі). Потім прочитайте вміст цього файлу і помістіть в інший словник. Виведіть на екран прочитані дані (словник).

Завдання №2. Напишіть вираз-генератор, який повертав би площі кіл з радіусами в діапазоні від 1 до 100 із кроком 0.1. Значення мають бути округлені до сотих. Вивести на екран перші п'ять значень.

Завдання №3. Створіть функцію-генератор, яка повертала б значення деякої функції $f(x)$ для значень $x \in [a; b]$ з кроком 0,01. Величини a, b, і навіть функцію f слід передавати аргументами у функцію-генератор. Викличте функцію-генератор зі значеннями $a = -20;\ b = 100$ для функції $f(x) = -1{,}5x + 2$, яку визначте як лямбда-функцію безпосередньо у аргументі функції-генератора. Виведіть на екран перші 20 значень, повернуті функцією-генератором. Будь-які колекції у програмі не використовувати.

Завдання №4. Вводяться назви міст в один рядок через пробіл. Необхідно за допомогою функції `filter` отримати ітератор, який повертав би тільки назви довжиною більше 5 символів. Отримайте перші три значення (або менше, якщо значень менше трьох) на виході цієї функції за допомогою `next` та відобразіть їх на екрані.

Варіант №8

Завдання №1. Імпортуйте в програму стандартний модуль `random` і, використовуючи функцію `randint`, сформуйте двовимірний список розміром 5x5 елементів з випадковими значеннями в діапазоні [-5; 5]. Двовимірний список слід сформувати за допомогою вкладених генераторів списків. Збережіть отримані дані файлі з бінарним режимом доступу. Потім прочитайте з нього дані і помістіть в інший список. Виведіть отриманий двовимірний список.

Завдання №2. Напишіть вираз-генератор, який повертав би кількості діагоналей n-кутника, де n змінюється від 1 до 1000. Виведіть на екран перші 15 значень. Для n = 1, 2, 3 генератор має повертати 0.

Завдання №3. Є рядок `symbols`, який містить довільний набір різних символів (наприклад `symbols = "abc1234#$%"`) довжиною не менше восьми символів. Необхідно записати вираз-генератор, який би формував та видавав усі *невпорядковані* комбінації із трьох знаків (*невпорядкованість* означає, що, наприклад, поєднання `"b14"` і `"4b1"` ми не розрізняємо і видаватись має тільки одна подібна комбінація). Вивести перші 50 згенерованих комбінацій на екран.

Завдання №4. В зовнішньому файлі `matrix.txt` записана квадратна матриця M, елементами якої є цілі числа. Файл створіть самостійно. Приклад вмісту файлу:

```
2 4 7
7 0 2
8 4 2
```

Програма повинна прочитати цю матрицю у двовимірний кортеж або список. Після цього програма створює новий двовимірний кортеж або список, який містить нову матрицю, що є результатом повороту матриці M на 90 градусів за годинниковою стрілкою. Таким чином, для наведеного прикладу результат має виглядати так:

```
8 7 2
4 0 4
2 2 7
```

Відобразити нову матрицю на екрані. Поворот матриці бажано виконати за допомогою функцій `zip` і `map` без використання циклів.

Варіант №9

Завдання №1. Імпортуйте в програму стандартний модуль `math` та, використовуючи функцію `sin`, обчисліть синус для значень від $-\pi$ до π з кроком $\pi/10$. Результат подайте у вигляді списку. Сформуйте цей список за допомогою генератора списків. Отриманий список збережіть у файлі у бінарному режимі доступу. Потім прочитайте вміст цього файлу і збережіть його в іншому списку. На екрані виведіть прочитані дані (останній список).

Завдання №2. Записати вираз-генератор, який видає значення лінійної функції $f(x) = 0{,}5\sqrt{x} - 2$ в діапазоні $x \in [0; 100]$ з кроком 0,01 (для x). Значення мають бути округлені до сотих. Вивести перші 50 згенерованих значень.

Завдання №3. Написати функцію-генератор, яка має видавати значення арифметичної прогресії: $a_n = a_0 + d(n-1)$. Значення a_0 і d вказуються як параметри цієї функції. Викличте цю функцію зі значеннями аргументів $a_0 = -0.5$ і $d = 0.1$ та виведіть на екран перші 50 значень, які повертає цей генератор. Будь-які колекції у програмі не використовувати.

Завдання №4. Вводяться два списки цілих чисел. Необхідно попарно перебрати їх елементи та перемножити між собою. Під час реалізації програми використовуйте функції `zip` та `map`. Виведіть на екран перші три значення за допомогою функції `next`. (Вважається, що кожен з вихідних списків містить принаймні три значення).

Варіант №10

Завдання №1. Імпортуйте стандартний модуль `time`, використовуючи однойменну функцію `time` з цього модуля, визначте час роботи програми, яка зберігає у файл таблицю множення цілих чисел від 1 до 10 (кожен

множиться від 1 до 10). Потім, відкриває цей файл на читання і формує з прочитаних даних двовимірний кортеж у форматі: ((2, 1, 2), (2, 2, 4), (2, 3, 6), …). Виведіть на екран зчитаний кортеж та час роботи програми.

Завдання №2. Є список із нот:

```
notes = ['до', 'ре', 'мі', 'фа', 'соль', 'ля', 'сі']
```

Необхідно записати генератор, який випадковим чином видавав би ноти з цього списку 1 000 000 разів. Для випадкового вибору нот, використовуйте функцію `choice` модуля `random`. Виведіть на екран перші 20 значень цього генератора.

Завдання №3. Визначте функцію-генератор, яка нескінченно повертала б двійкові подання натуральних чисел (як об'єкти `str`), починаючи від числа a, яке задається як аргумент функції. Викличте цю функцію зі значенням аргументу $a = 5$ і виведіть перші 20 чисел, повернутих цією функцією-генератором. Будь-які колекції у програмі не використовувати.

Завдання №4. Вводяться назви міст в один рядок через пробіл. Необхідно за допомогою функції `map`, перетворити цей рядок на список, який містить лише назви міст довжиною понад 5 символів, а решта назв має бути замінена на символ дефісу (`"-"`). Сформувати список із отриманих значень та відобразити його на екрані.

Зміст звіту

1. Титульний лист із назвою лабораторної роботи, номером свого варіанту, прізвищем студента та групи.
2. Мета лабораторної роботи.
3. Завдання.
4. Лістинг програми (скопійований з IDE).
5. Результати роботи програми (скріншоти).